e	01100101	g
l	01101100	h
e	01100101	o
c	01100011	s
t	01110100	t
r	01110010	w
i	01101001	r
c	01100011	i
i	01101001	t
t	01110100	e
y	01111001	r

Letter 1

```
00000010 (Hex: 02)
⊕ 01100111 (g)
----------
01100101 = e
```

Letter 2

```
00000100 (Hex: 04)
⊕ 01101000 (h)
----------
01101100 = l
```

Letter 3

```
00001010 (Hex: 0A)

⊕ 01101111 (o)

----------

01100101 = e
```

Letter 4

```
00010000 (Hex: 10)

⊕ 01110011 (s)

----------

01100011 = c
```

Letter 5

```
00000000 (Hex: 00)
⊕ 01110100 (t)
----------
01110100 = t
```

Letter 6

```
00000101 (Hex: 05)
⊕ 01110111 (w)
----------
01110010 = r
```

Letter 7

```
00011011 (Hex: 1B)
⊕ 01110010 (r)
----------
01101001 = i
```

Letter 8

```
00001010 (Hex: 0A)
⊕ 01101001 (i)
----------
01100011 = c
```

Letter 9

```
00011101 (Hex: 1D)
⊕ 01110100 (t)
----------
01101001 = i
```

Letter 10

```
00010001 (Hex: 11)
⊕ 01100101 (e)
----------
01110100 = t
```

Letter 11

```
00001011 (Hex: 0B)
```

⊕ 01110010 (r)

01111001 = y

1. My Plaintext: electricity

↓

Convert letters to binary using ASCII

↓

Use my key: ghostwriter

↓

XOR the plaintext and key together

↓

Get the encrypted binary values

↓

Convert binary values to hexadecimal

↓

Ciphertext

↓

XOR the ciphertext with the same key

↓

Convert the binary back to ASCII

↓

Final Result: electricity

2. XOR encryption can still work when the plaintext and key are different lengths by repeating the key until it matches the length of the message. If the plaintext is longer than the key, the key starts over from the beginning and continues repeating until every character in the plaintext has a matching key character. The XOR operation is then performed on each pair of characters. While this method works, it is not as secure as using

a key that is the same length as the plaintext because repeating patterns in the key can make the encrypted message easier to analyze and break.

3.Whole thing was a challenge, painful on the brain, literally nothing was even clicking for me at first, couple YouTube videos and google searches later I started to get it down but not satisfied with how long this took me

4. To encrypt my message, I first chose the word electricity as the plaintext and ghostwriter as the secret key. Since XOR encryption requires the key to be the same length as the message, both words have 11 letters, so they work together. I started by converting every letter in both words into its 8-bit ASCII binary value. The letter e becomes 01100101 and g becomes 01100111. After converting everything to binary, I used the XOR operation to compare each bit from the plaintext with the matching bit from the key. If the bits were different, the result was 1, and if they were the same, the result was 0. For the first letter, e XOR g produced 00000010. Once I had the binary result for each letter, I converted those binary values into hexadecimal because hexadecimal is shorter and easier to read. For example, 00000010 becomes 02 in hexadecimal. Repeating this process for every letter created the final encrypted ciphertext, which could then be decrypted later by performing the XOR operation again using the same secret key.

5. To decrypt my message, I started with the hexadecimal ciphertext that was created during the encryption stage. Since computers work with binary, I converted each hexadecimal value back into an 8-bit binary number. Like the ciphertext value, 02 becomes 00000010. Then I used the same secret key, ghostwriter, and converted each letter into its binary ASCII value. I then performed the XOR operation again between the ciphertext and the matching key character. The first letter, 00000010 XOR 01100111, resulted in 01100101. The final step was converting the binary result back into a character using the ASCII table. 01100101 translates to the letter e. I repeated this process for the rest of the ciphertext, and the result spelled out electricity, which matched the original plaintext message. This confirmed that the encryption and decryption process worked correctly.